INSTITUT SUPÉRIEUR DE GESTION DE TUNIS
Since 1969

RAPPORT DU PROJET POO

GESTION D'UN AGENDA PERSONNEL

NIVEAU D'ÉTUDE : 2 ÈRE ANNÉE LICENCE NATIONALE
FILIÈRE : INFORMATIQUE DE GESTION
GROUPE N°:1

RAPPORT PRÉSENTÉ PAR :
AMIRA MIDANI & EYA IBN ABDELHEDI

ANNÉE UNIVERSITAIRE 2025/2026
2ÉME SEMESTRE

INTRODUCTION :

Ce rapport présente le travail réalisé dans le cadre du TP de développement Java. L'objectif est de concevoir et développer une application de gestion d'un agenda personnel, dotée d'une interface graphique permettant à l'utilisateur de gérer ses événements de manière intuitive.

L'application permet d'effectuer les opérations CRUD (Création, Lecture, Mise à jour, Suppression) sur les événements, de rechercher par différents critères, et d'exporter les données vers un fichier texte. L'ensemble des données est stocké dans une base de données relationnelle Oracle.

Ce projet met en pratique les concepts étudiés en cours, notamment : JDBC pour la connexion à la base de données, la gestion des exceptions, les classes internes anonymes, et les flux d'entrées/sorties sur fichiers.

ENVIRONNEMENT TECHNIQUE

1. Outils utilisés

Outil	Description
NetBeans IDE 28	Environnement de développement intégré
JDK 25	Kit de développement Java
JavaFX	Bibliothèque pour l'interface graphique
Oracle XE 21c	Base de données relationnelle (via Docker)
SQL Developer	Client graphique pour Oracle
ojdbc11.jar	Pilote JDBC pour Oracle
Docker Desktop	Conteneurisation de la base de données

2. Architecture du projet

Le projet suit le modèle d'architecture en couches avec séparation des responsabilités :

Fichier	Rôle	Couche
ConnexionDB.java	Connexion à Oracle avec JDBC	Accès aux données
Evenement.java	Modèle de données	Métier
EvenementDAO.java	Opérations CRUD + recherche	Accès aux données
AgendaApp.java	Interface graphique JavaFX	Présentation

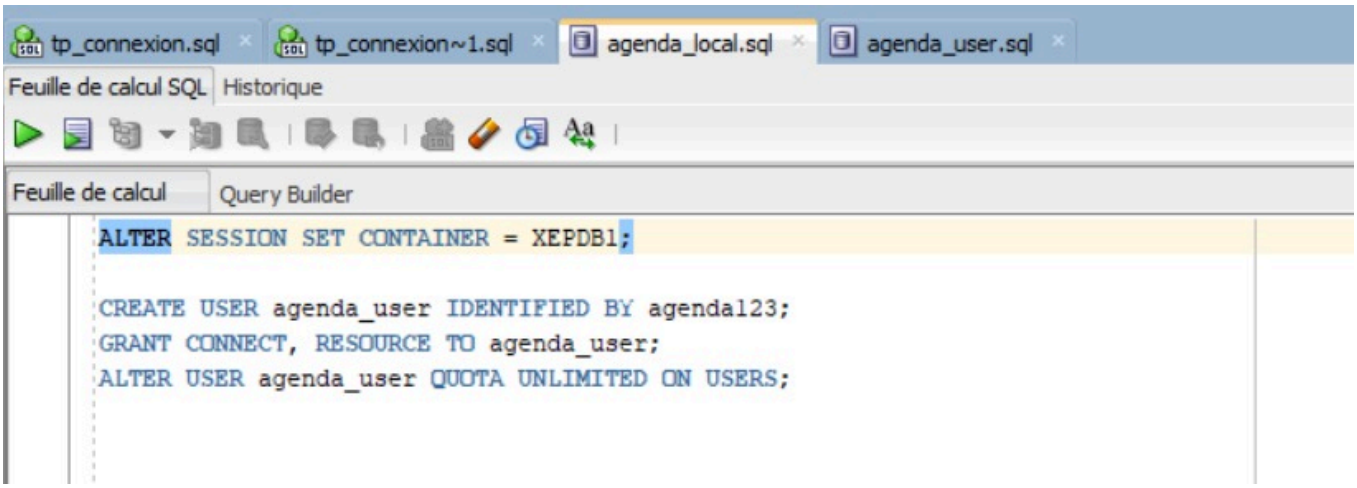
BASE DE DONNÉES ORACLE

1. Installation via Docker

La base de données Oracle XE a été déployée via Docker avec la commande suivante :

```
C:\Users\USER>docker run -d -p 1521:1521 -p 5500:5500 -e ORACLE_PASSWORD=oracle --name oracle-db gvenzl/oracle-xe
docker: Error response from daemon: Conflict. The container name "/oracle-db" is already in use by container "be87484565
90d1d17d0fe5d53c7d6b86e6a1f7e40ee3c3681731feecc9524dd4". You have to remove (or rename) that container to be able to reu
se that name.
```

2. Création de l'utilisateur



3. Structure de la table

```
CREATE TABLE evenement (
    id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    titre VARCHAR2(100) NOT NULL,
    date_evenement DATE NOT NULL,
    heure VARCHAR2(5) NOT NULL,
    description VARCHAR2(500),
    type_evenement VARCHAR2(50) NOT NULL
);
ALTER TABLE evenement ADD important NUMBER(1) DEFAULT 0;
```

La colonne id est générée automatiquement par Oracle (IDENTITY), ce qui garantit l'unicité de chaque événement.

4. Paramètres de connexion

Paramètre	Valeur
Host	localhost
Port	1521
Service Name	XEPDB1
Utilisateur	agenda_user
Mot de passe	agenda123

CONNEXION À LA BASE (JDBC)

CLASSE CONNEXIONDB

```java
package agendaapp;

import java.sql.*;

public class ConnexionDB {
    private static final String URL = "jdbc:oracle:thin:@localhost:1521/XEPDB1";
    private static final String USER = "agenda_user";
    private static final String PASSWORD = "agenda123";

    public static Connection getConnection() throws SQLException {
        try {
            Class.forName("oracle.jdbc.driver.OracleDriver");
        } catch (ClassNotFoundException e) {
            System.out.println("Impossible de charger le pilote JDBC !");
        }
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

Cette classe gère la connexion à la base de données Oracle en utilisant l'API JDBC . Elle utilise :

- Class.forName() pour charger le pilote JDBC Oracle
- DriverManager.getConnection() pour établir la connexion.
- Un bloc try/catch pour gérer l'exception ClassNotFoundException

CLASSE EVENEMENT

Cette classe représente un événement de l'agenda. C'est un POJO (Plain Old Java Object) avec des attributs privés et des accesseurs (getters/setters). Chaque événement possède :

ATTRIBUTS :

```java
public class Evenement {
    private int id;
    private String titre;
    private String dateEvenement;
    private String heure;
    private String description;
    private String typeEvenement;
```

```java
public Evenement() {}

public Evenement(String titre, String dateEvenement, String heure, String description, String typeEvenement) {
    this.titre = titre;
    this.dateEvenement = dateEvenement;
    this.heure = heure;
    this.description = description;
    this.typeEvenement = typeEvenement;
}

public int getId() { return id; }
public void setId(int id) { this.id = id; }
```

```java
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getTitre() { return titre; }
    public void setTitre(String titre) { this.titre = titre; }

    public String getDateEvenement() { return dateEvenement; }
    public void setDateEvenement(String dateEvenement) { this.dateEvenement = dateEvenement; }

    public String getHeure() { return heure; }
    public void setHeure(String heure) { this.heure = heure; }

    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }

    public String getTypeEvenement() { return typeEvenement; }
    public void setTypeEvenement(String typeEvenement) { this.typeEvenement = typeEvenement; }
}
```

COUCHE DAO (DATA ACCESS OBJECT)

```java
    public boolean isImportant() { return important; }
    public void setImportant(boolean important) { this.important = important; }

    public String getImportantText() { return important ? "* Oui" : "Non"; }
}
```

CLASSE EVENEMENTDAO

Cette classe implémente le patron DAO (Data Access Object). Elle contient toutes les méthodes d'accès à la base de données en utilisant JDBC :

MÉTHODES CRUD :

AJOUTER UN ÉVÉNEMENT

```java
package agendaapp;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class EvenementDAO {

    public void ajouter(Evenement e) throws SQLException {
        String sql = "INSERT INTO evenement (titre, date_evenement, heure, description, type_evenement) VALUES (?, TO_DATE
        Connection conn = ConnexionDB.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql);
        ps.setString(1, e.getTitre());
        ps.setString(2, e.getDateEvenement());
        ps.setString(3, e.getHeure());
        ps.setString(4, e.getDescription());
        ps.setString(5, e.getTypeEvenement());
        ps.executeUpdate();
        ps.close();
        conn.close();
    }
```

Insère un événement avec PreparedStatement et INSERT INTO

MODIFIER UN ÉVÉNEMENT

```java
public void modifier(Evenement e) throws SQLException {
    String sql = "UPDATE evenement SET titre=?, date_evenement=TO_DATE(?, 'YYYY-MM-DD'), heure=?, description=?, type_
    Connection conn = ConnexionDB.getConnection();
    PreparedStatement ps = conn.prepareStatement(sql);
    ps.setString(1, e.getTitre());
    ps.setString(2, e.getDateEvenement());
    ps.setString(3, e.getHeure());
    ps.setString(4, e.getDescription());
    ps.setString(5, e.getTypeEvenement());
    ps.setInt(6, e.getId());
    ps.executeUpdate();
    ps.close();
    conn.close();
}
```

Met à jour un événement avec UPDATE et PreparedStatement

COUCHE DAO (DATA ACCESS OBJECT)

SUPPRIMER

```
public void supprimer(int id) throws SQLException {
    String sql = "DELETE FROM evenement WHERE id = ?";
    Connection conn = ConnexionDB.getConnection();
    PreparedStatement ps = conn.prepareStatement(sql);
    ps.setInt(1, id);
    ps.executeUpdate();
    ps.close();
    conn.close();
}
```

Supprime un événement par son ID avec DELETE

AFFICHER TOUS

```
public List<Evenement> afficherTous() throws SQLException {
    List<Evenement> list = new ArrayList<>();
    String sql = "SELECT id, titre, TO_CHAR(date_evenement, 'YYYY-MM-DD') AS date_evenement, heure, description, type_
    Connection conn = ConnexionDB.getConnection();
    Statement st = conn.createStatement();
    ResultSet rs = st.executeQuery(sql);
    while (rs.next()) {
        Evenement e = new Evenement();
        e.setId(rs.getInt("id"));
        e.setTitre(rs.getString("titre"));
        e.setDateEvenement(rs.getString("date_evenement"));
        e.setHeure(rs.getString("heure"));
        e.setDescription(rs.getString("description"));
        e.setTypeEvenement(rs.getString("type_evenement"));
        list.add(e);
    }
    rs.close();
    st.close();
    conn.close();
    return list;
}
```

Récupère tous les événements avec SELECT et TO_CHAR

COUCHE DAO (DATA ACCESS OBJECT)

MÉTHODES DE RECHERCHE:

RECHERCHERPARTITRE()

```
public List<Evenement> rechercherParTitre(String titre) throws SQLException {
    List<Evenement> list = new ArrayList<>();
    String sql = "SELECT id, titre, TO_CHAR(date_evenement, 'YYYY-MM-DD') AS date_evenement, heure, description, type_
    Connection conn = ConnexionDB.getConnection();
    PreparedStatement ps = conn.prepareStatement(sql);
    ps.setString(1, "%" + titre.toLowerCase() + "%");
    ResultSet rs = ps.executeQuery();
    while (rs.next()) {
        Evenement e = new Evenement();
        e.setId(rs.getInt("id"));
        e.setTitre(rs.getString("titre"));
        e.setDateEvenement(rs.getString("date_evenement"));
        e.setHeure(rs.getString("heure"));
        e.setDescription(rs.getString("description"));
        e.setTypeEvenement(rs.getString("type_evenement"));
        list.add(e);
    }
    rs.close();
    ps.close();
    conn.close();
    return list;
}
```

Recherche avec LIKE et LOWER() pour ignorer la casse

RECHERCHERPARDATE()

```
public List<Evenement> rechercherParDate(String date) throws SQLException {
    List<Evenement> list = new ArrayList<>();
    String sql = "SELECT id, titre, TO_CHAR(date_evenement, 'YYYY-MM-DD') AS date_evenement, heure, description, type_
    Connection conn = ConnexionDB.getConnection();
    PreparedStatement ps = conn.prepareStatement(sql);
    ps.setString(1, date);
    ResultSet rs = ps.executeQuery();
    while (rs.next()) {
        Evenement e = new Evenement();
        e.setId(rs.getInt("id"));
        e.setTitre(rs.getString("titre"));
        e.setDateEvenement(rs.getString("date_evenement"));
        e.setHeure(rs.getString("heure"));
        e.setDescription(rs.getString("description"));
        e.setTypeEvenement(rs.getString("type_evenement"));
        list.add(e);
    }
    rs.close();
    ps.close();
    conn.close();
    return list;
}
```

Recherche par date exacte avec TO_DATE

COUCHE DAO (DATA ACCESS OBJECT)

RECHERCHERPARTYPE()

```java
public List<Evenement> rechercherParType(String type) throws SQLException {
    List<Evenement> list = new ArrayList<>();
    String sql = "SELECT id, titre, TO_CHAR(date_evenement, 'YYYY-MM-DD') AS date_evenement, heure, description, type_
    Connection conn = ConnexionDB.getConnection();
    PreparedStatement ps = conn.prepareStatement(sql);
    ps.setString(1, "%" + type.toLowerCase() + "%");
    ResultSet rs = ps.executeQuery();
    while (rs.next()) {
        Evenement e = new Evenement();
        e.setId(rs.getInt("id"));
        e.setTitre(rs.getString("titre"));
        e.setDateEvenement(rs.getString("date_evenement"));
        e.setHeure(rs.getString("heure"));
        e.setDescription(rs.getString("description"));
        e.setTypeEvenement(rs.getString("type_evenement"));
        list.add(e);
    }
    rs.close();
    ps.close();
    conn.close();
    return list;
}
```

Recherche par catégorie d'événement

MÉTHODES DE VALIDATION:

EXISTEDOUBLON()

```java
public boolean existeDoublon(String titre, String date) throws SQLException {
    String sql = "SELECT COUNT(*) FROM evenement WHERE LOWER(titre) = ? AND date_evenement = TO_DATE(?, 'YYYY-MM-DD')"
    Connection conn = ConnexionDB.getConnection();
    PreparedStatement ps = conn.prepareStatement(sql);
    ps.setString(1, titre.toLowerCase());
    ps.setString(2, date);
    ResultSet rs = ps.executeQuery();
    rs.next();
    int count = rs.getInt(1);
    rs.close();
    ps.close();
    conn.close();
    return count > 0;
}
```

Vérifie si un événement avec le même titre et date existe

EXISTECONFLIT()

```java
public boolean existeConflit(String date, String heure) throws SQLException {
    String sql = "SELECT COUNT(*) FROM evenement WHERE date_evenement = TO_DATE(?, 'YYYY-MM-DD') AND heure = ?";
    Connection conn = ConnexionDB.getConnection();
    PreparedStatement ps = conn.prepareStatement(sql);
    ps.setString(1, date);
    ps.setString(2, heure);
    ResultSet rs = ps.executeQuery();
    rs.next();
    int count = rs.getInt(1);
    rs.close();
    ps.close();
    conn.close();
    return count > 0;
}
```


Vérifie si un événement existe à la même date et heure

COUCHE DAO (DATA ACCESS OBJECT)

EXISTECONFLITMODIFICATION()

```java
public boolean existeConflitModification(String date, String heure, int idExclu) throws SQLException {
    String sql = "SELECT COUNT(*) FROM evenement WHERE date_evenement = TO_DATE(?, 'YYYY-MM-DD') AND heure = ? AND id
    Connection conn = ConnexionDB.getConnection();
    PreparedStatement ps = conn.prepareStatement(sql);
    ps.setString(1, date);
    ps.setString(2, heure);
    ps.setInt(3, idExclu);
    ResultSet rs = ps.executeQuery();
    rs.next();
    int count = rs.getInt(1);
    rs.close();
    ps.close();
    conn.close();
    return count > 0;
}
```

Vérifie les conflits en excluant l'événement en cours

MÉTHODE D'EXPORT :

La méthode exporterFichier() utilise les classes PrintWriter et FileWriter du package java.io pour écrire les événements dans un fichier texte :

```java
public void exporterFichier(String nomFichier, List<Evenement> liste) throws java.io.IOException {
    java.io.PrintWriter pw = new java.io.PrintWriter(new java.io.FileWriter(nomFichier));
    pw.println("========================================");
    pw.println("       AGENDA PERSONNEL - EXPORT");
    pw.println("========================================");
    pw.println();
    for (Evenement e : liste) {
        pw.println("ID          : " + e.getId());
        pw.println("Titre       : " + e.getTitre());
        pw.println("Date        : " + e.getDateEvenement());
        pw.println("Heure       : " + e.getHeure());
        pw.println("Description : " + e.getDescription());
        pw.println("Type        : " + e.getTypeEvenement());
        pw.println("----------------------------------------");
    }
    pw.println("Nombre total d'événements : " + liste.size());
    pw.close();
}
}
```

COUCHE DAO (DATA ACCESS OBJECT)

MÉTHODE : AFFICHERIMPORTANTS() :

```
private void afficherImportants() {
    try {
        List<Evenement> list = dao.afficherImportants();
        if (list.isEmpty()) {
            alerte(Alert.AlertType.INFORMATION, "Aucun événement important trouvé.");
        }
        table.setItems(FXCollections.observableArrayList(list));
    } catch (SQLException ex) {
        alerte(Alert.AlertType.ERROR, "Erreur: " + ex.getMessage());
    }
}
```

Cette méthode permet de récupérer uniquement les événements marqués comme importants depuis la base de données et de les convertir en objets Evenement.

Classe principale AgendaApp

```
public class AgendaApp extends Application {

    private TableView<Evenement> table;
    private TextField txtTitre, txtDate, txtHeure, txtDescription;
    private ComboBox<String> comboType;
    private TextField txtRecherche;
    private ComboBox<String> comboRecherche;
    private EvenementDAO dao = new EvenementDAO();

    @Override
    public void start(Stage primaryStage) {
        primaryStage.setTitle("📅 Agenda Personnel");
```

Cette classe étend Application de JavaFX et constitue le point d'entrée de l'application. Elle contient :

- **Header**

```
// === HEADER ===
Label header = new Label("📅 Mon Agenda Personnel");
header.setFont(Font.font("Segoe UI", FontWeight.BOLD, 24));
header.setStyle("-fx-text-fill: white;");
HBox headerBox = new HBox(header);
headerBox.setAlignment(Pos.CENTER);
headerBox.setPadding(new Insets(15));
headerBox.setStyle("-fx-background-color: linear-gradient(to right, #2c3e50, #3498db); -fx-background-radius: 8;");
```

Titre de l'application avec dégradé bleu

- **Formulaire**

```
// === FORMULAIRE ===
Label lblFormTitle = new Label("Nouvel Événement");
lblFormTitle.setFont(Font.font("Segoe UI", FontWeight.BOLD, 16));
lblFormTitle.setStyle("-fx-text-fill: #2c3e50;");

txtTitre = createStyledTextField("Titre de l'événement");
txtDate = createStyledTextField("2026-05-10");
txtHeure = createStyledTextField("14:00");
txtDescription = createStyledTextField("Description de l'événement");

comboType = new ComboBox<>();
comboType.getItems().addAll("Réunion", "Personnel", "Travail", "Médical", "Autre");
comboType.setValue("Réunion");
comboType.setStyle("-fx-font-size: 13px; -fx-pref-width: 250;");

GridPane form = new GridPane();
form.setHgap(15);
form.setVgap(12);
form.add(createLabel("Titre"), 0, 0);            form.add(txtTitre, 1, 0);
form.add(createLabel("Date (YYYY-MM-DD)"), 0, 1); form.add(txtDate, 1, 1);
form.add(createLabel("Heure (HH:MM)"), 0, 2);    form.add(txtHeure, 1, 2);
form.add(createLabel("Description"), 0, 3);       form.add(txtDescription, 1, 3);
```

INTERFACE GRAPHIQUE (JAVAFX)

```
form.add(createLabel("Type"), 0, 4);                form.add(comboType, 1, 4);

VBox formBox = new VBox(10, lblFormTitle, form);
formBox.setPadding(new Insets(15));
formBox.setStyle("-fx-background-color: white; -fx-background-radius: 8; -fx-effect: dropshadow(three-pass-box, rg

// === BOUTONS CRUD ===
```

GridPane avec TextField, ComboBox pour la saisie

- **Boutons CRUD**

```
// === BOUTONS CRUD ===
Button btnAjouter = createStyledButton("➕ Ajouter", "#27ae60");
Button btnModifier = createStyledButton("✏ Modifier", "#f39c12");
Button btnSupprimer = createStyledButton("🗑 Supprimer", "#e74c3c");
Button btnEffacer = createStyledButton("🔄 Effacer", "#95a5a6");
Button btnExporter = createStyledButton("📄 Exporter", "#2980b9");

btnAjouter.setOnAction(e -> ajouterEvenement());
btnModifier.setOnAction(e -> modifierEvenement());
btnSupprimer.setOnAction(e -> supprimerEvenement());
btnEffacer.setOnAction(e -> effacerFormulaire());
btnExporter.setOnAction(e -> exporterEvenements());

HBox boutons = new HBox(10, btnAjouter, btnModifier, btnSupprimer, btnEffacer, btnExporter);
boutons.setAlignment(Pos.CENTER);
boutons.setPadding(new Insets(10));
```

Ajouter, Modifier, Supprimer, Effacer, Exporter (colorés et stylisés)

- **Recherche**

```
// === RECHERCHE ===
Label lblRechTitle = new Label("🔍 Recherche");
lblRechTitle.setFont(Font.font("Segoe UI", FontWeight.BOLD, 16));
lblRechTitle.setStyle("-fx-text-fill: #2c3e50;");

txtRecherche = createStyledTextField("Mot clé...");
comboRecherche = new ComboBox<>();
comboRecherche.getItems().addAll("Titre", "Date", "Type");
comboRecherche.setValue("Titre");
comboRecherche.setStyle("-fx-font-size: 13px;");

Button btnRechercher = createStyledButton("🔍 Rechercher", "#3498db");
Button btnAfficherTous = createStyledButton("📋 Afficher Tous", "#8e44ad");

btnRechercher.setOnAction(e -> rechercher());
btnAfficherTous.setOnAction(e -> chargerTous());

HBox rechercheRow = new HBox(10, txtRecherche, comboRecherche, btnRechercher, btnAfficherTous);
rechercheRow.setAlignment(Pos.CENTER_LEFT);

VBox rechercheBox = new VBox(10, lblRechTitle, rechercheRow);
rechercheBox.setPadding(new Insets(15));
rechercheBox.setStyle("-fx-background-color: white; -fx-background-radius: 8; -fx-effect: dropshadow(three-pass-bo
```

Barre de recherche avec filtre par titre, date ou type

INTERFACE GRAPHIQUE (JAVAFX)

- **TableView**

```
// === TABLEAU ===
table = new TableView<>();
table.setStyle("-fx-font-size: 13px;");

TableColumn<Evenement, Integer> colId = new TableColumn<>("ID");
colId.setCellValueFactory(new PropertyValueFactory<>("id"));
colId.setPrefWidth(50);
colId.setStyle("-fx-alignment: CENTER;");

TableColumn<Evenement, String> colTitre = new TableColumn<>("Titre");
colTitre.setCellValueFactory(new PropertyValueFactory<>("titre"));
colTitre.setPrefWidth(150);

TableColumn<Evenement, String> colDate = new TableColumn<>("Date");
colDate.setCellValueFactory(new PropertyValueFactory<>("dateEvenement"));
colDate.setPrefWidth(120);
colDate.setStyle("-fx-alignment: CENTER;");

TableColumn<Evenement, String> colHeure = new TableColumn<>("Heure");
colHeure.setCellValueFactory(new PropertyValueFactory<>("heure"));
colHeure.setPrefWidth(80);
colHeure.setStyle("-fx-alignment: CENTER;");
```

Tableau affichant tous les événements avec 6 colonnes

```
TableColumn<Evenement, String> colDesc = new TableColumn<>("Description");
colDesc.setCellValueFactory(new PropertyValueFactory<>("description"));
colDesc.setPrefWidth(200);

TableColumn<Evenement, String> colType = new TableColumn<>("Type");
colType.setCellValueFactory(new PropertyValueFactory<>("typeEvenement"));
colType.setPrefWidth(120);
colType.setStyle("-fx-alignment: CENTER;");

table.getColumns().addAll(colId, colTitre, colDate, colHeure, colDesc, colType);
table.setColumnResizePolicy(TableView.CONSTRAINED_RESIZE_POLICY);
table.setPrefHeight(350);

table.getSelectionModel().selectedItemProperty().addListener((obs, oldVal, newVal) -> {
    if (newVal != null) {
        txtTitre.setText(newVal.getTitre());
        txtDate.setText(newVal.getDateEvenement());
        txtHeure.setText(newVal.getHeure());
        txtDescription.setText(newVal.getDescription());
        comboType.setValue(newVal.getTypeEvenement());
    }
});
```

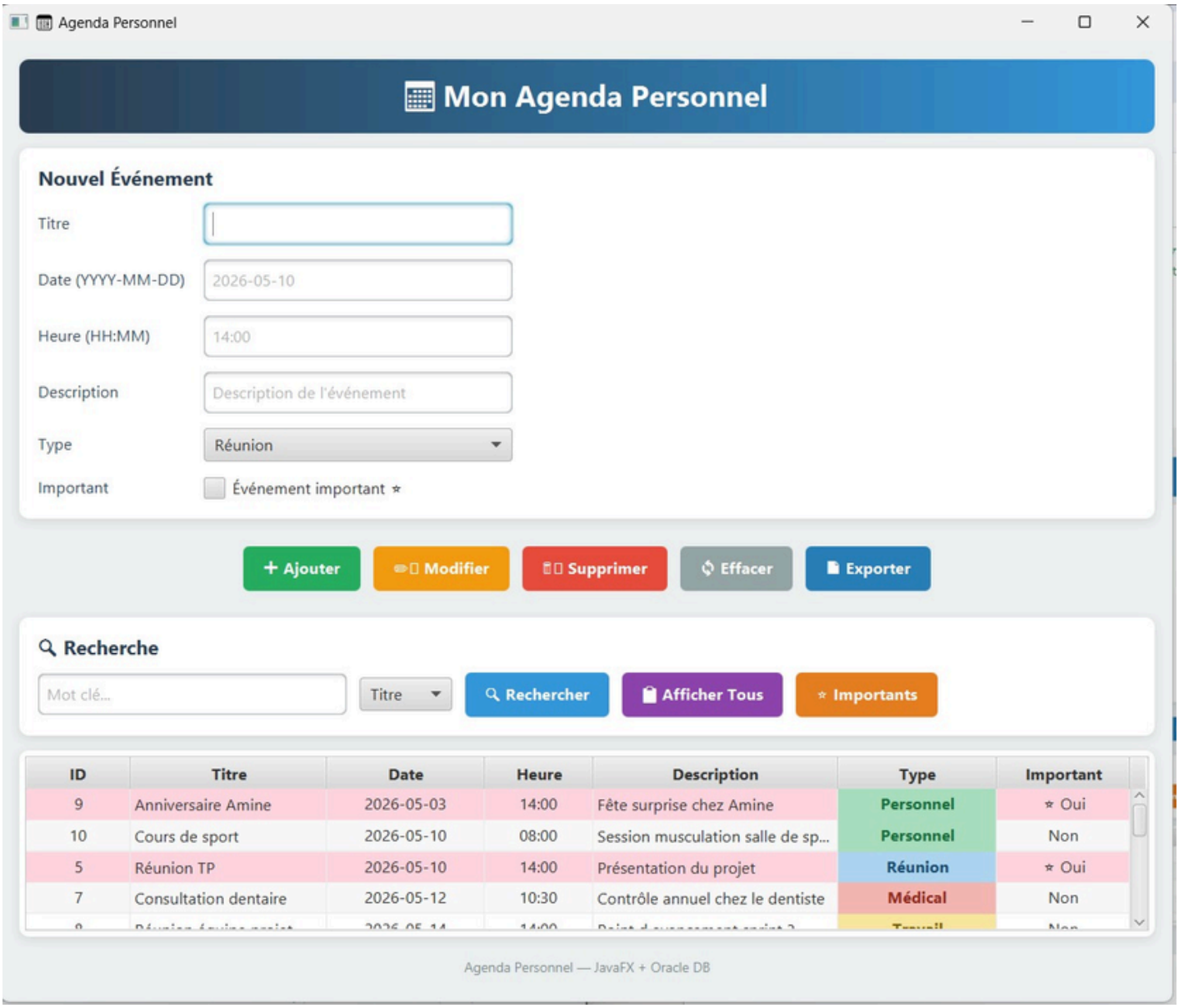
```
VBox tableBox = new VBox(table);
tableBox.setPadding(new Insets(5));
tableBox.setStyle("-fx-background-color: white; -fx-background-radius: 8; -fx-effect: dropshadow(three-pass-box, r
VBox.setVgrow(table, Priority.ALWAYS);
```

INTERFACE GRAPHIQUE (JAVAFX)

- **Footer**

```
// === FOOTER ===
Label footer = new Label("Agenda Personnel — JavaFX + Oracle DB");
footer.setStyle("-fx-text-fill: #7f8c8d; -fx-font-size: 11px;");
HBox footerBox = new HBox(footer);
footerBox.setAlignment(Pos.CENTER);
footerBox.setPadding(new Insets(5));
```

Signature de l'application



Classes internes anonymes :

Les événements des boutons utilisent des expressions lambda, qui sont une forme moderne des classes internes anonymes

```
btnAjouter.setOnAction(e -> ajouterEvenement());
btnModifier.setOnAction(e -> modifierEvenement());
btnSupprimer.setOnAction(e -> supprimerEvenement());
btnEffacer.setOnAction(e -> effacerFormulaire());
btnExporter.setOnAction(e -> exporterEvenements());
```

Méthode validerFormulaire()

```
private boolean validerFormulaire() {
    String titre = txtTitre.getText().trim();
    String date = txtDate.getText().trim();
    String heure = txtHeure.getText().trim();
    String description = txtDescription.getText().trim();

    // Mettre à jour les champs avec les valeurs nettoyées (sans espaces)
    txtTitre.setText(titre);
    txtDate.setText(date);
    txtHeure.setText(heure);
    txtDescription.setText(description);

    // 1. Champs obligatoires
    if (titre.isEmpty() || date.isEmpty() || heure.isEmpty()) {
        alerte(Alert.AlertType.WARNING, "Remplissez au moins le titre, la date et l'heure.");
        return false;
    }

    // 2. Titre trop court
    if (titre.length() < 3) {
        alerte(Alert.AlertType.WARNING, "Le titre doit contenir au moins 3 caractères.");
        return false;
    }
```

```
    // 3. Titre trop long (max 100 caractères dans la BD)
    if (titre.length() > 90) {
        alerte(Alert.AlertType.WARNING, "Le titre ne doit pas dépasser 90 caractères.\nActuellement : " + titre.length
        return false;
    }

    // 4. Titre contient uniquement des caractères valides
    if (!titre.matches("[a-zA-ZÀ-ÿ0-9 '\\-]+")) {
        alerte(Alert.AlertType.WARNING, "Le titre contient des caractères non autorisés.\nUtilisez uniquement des lett
        return false;
    }

    // 5. Description trop longue (max 500 caractères dans la BD)
    if (description.length() > 500) {
        alerte(Alert.AlertType.WARNING, "La description ne doit pas dépasser 500 caractères.\nActuellement : " + descr
        return false;
    }

    // 6. Format de date (YYYY-MM-DD)
    if (!validerDate(date)) {
        alerte(Alert.AlertType.WARNING, "Format de date invalide. Utilisez YYYY-MM-DD (ex: 2026-05-10).");
        return false;
    }
```

VALIDATION DES DONNÉES

Contrôle	Message d'erreur
Champs obligatoires	Remplissez au moins le titre, la date et l'heure
Titre minimum 3 caractères	Le titre doit contenir au moins 3 caractères
Titre maximum 90 caractères	Le titre ne doit pas dépasser 90 caractères
Caractères spéciaux interdits	Le titre contient des caractères non autorisés
Description max 500	La description ne doit pas dépasser 500
Format date YYYY-MM-DD	Format de date invalide
Format heure HH:MM	Format d'heure invalide
Date pas dans le passé	interdit

Validation date :

```
private boolean validerDate(String date) {
    try {
        LocalDate.parse(date, DateTimeFormatter.ofPattern("yyyy-MM-dd"));
        return true;
    } catch (DateTimeParseException e) {
        return false;
    }
}
```

Validation heure :

```
private boolean validerHeure(String heure) {
    if (!heure.matches("\\d{2}:\\d{2}")) return false;
    String[] parts = heure.split(":");
    int h = Integer.parseInt(parts[0]);
    int m = Integer.parseInt(parts[1]);
    return h >= 0 && h <= 23 && m >= 0 && m <= 59;
}
```

FONCTIONNALITÉS PRINCIPALES

1. GESTION DES ÉVÉNEMENTS (CRUD)

AJOUTER

```
private void ajouterEvenement() {
    if (!validerFormulaire()) return;
    try {
        if (dao.existeDoublon(txtTitre.getText(), txtDate.getText())) {
            alerte(Alert.AlertType.WARNING, "Cet événement existe déjà (même titre et même date) !");
            return;
        }
        if (dao.existeConflit(txtDate.getText(), txtHeure.getText())) {
            alerte(Alert.AlertType.WARNING, "Un événement existe déjà à cette date et heure !");
            return;
        }
        Evenement e = new Evenement(
            txtTitre.getText(), txtDate.getText(), txtHeure.getText(),
            txtDescription.getText(), comboType.getValue()
        );
        dao.ajouter(e);
        chargerTous();
        effacerFormulaire();
        alerte(Alert.AlertType.INFORMATION, "Événement ajouté avec succès !");
    } catch (SQLException ex) {
        alerte(Alert.AlertType.ERROR, "Erreur: " + ex.getMessage());
    }
```

- Créer un nouvel événement après validation complète

MODIFIER

```
private void modifierEvenement() {
    Evenement selected = table.getSelectionModel().getSelectedItem();
    if (selected == null) {
        alerte(Alert.AlertType.WARNING, "Sélectionnez un événement à modifier.");
        return;
    }
    if (!validerFormulaire()) return;
    try {
        if (dao.existeConflitModification(txtDate.getText(), txtHeure.getText(), selected.getId())) {
            alerte(Alert.AlertType.WARNING, "Un autre événement existe déjà à cette date et heure !");
            return;
        }
        selected.setTitre(txtTitre.getText());
        selected.setDateEvenement(txtDate.getText());
        selected.setHeure(txtHeure.getText());
        selected.setDescription(txtDescription.getText());
        selected.setTypeEvenement(comboType.getValue());
        dao.modifier(selected);
        chargerTous();
        effacerFormulaire();
        alerte(Alert.AlertType.INFORMATION, "Événement modifié avec succès !");
    } catch (SQLException ex) {
        alerte(Alert.AlertType.ERROR, "Erreur: " + ex.getMessage());
```

Sélectionner une ligne, modifier les champs, puis valider

FONCTIONNALITÉS PRINCIPALES

SUPPRIMER

```
private void supprimerEvenement() {
    Evenement selected = table.getSelectionModel().getSelectedItem();
    if (selected == null) {
        alerte(Alert.AlertType.WARNING, "Sélectionnez un événement à supprimer.");
        return;
    }
    Alert confirm = new Alert(Alert.AlertType.CONFIRMATION, "Supprimer cet événement ?");
    confirm.showAndWait().ifPresent(response -> {
        if (response == ButtonType.OK) {
            try {
                dao.supprimer(selected.getId());
                chargerTous();
                effacerFormulaire();
                alerte(Alert.AlertType.INFORMATION, "Événement supprimé !");
            } catch (SQLException ex) {
                alerte(Alert.AlertType.ERROR, "Erreur: " + ex.getMessage());
            }
        }
```

Sélectionner une ligne et confirmer la suppression

EFFACER

```
private void effacerFormulaire() {
    txtTitre.clear();
    txtDate.clear();
    txtHeure.clear();
    txtDescription.clear();
    comboType.setValue("Réunion");
    table.getSelectionModel().clearSelection();
}
```

Réinitialiser le formulaire sans toucher à la base

2 . RECHERCHE MULTICRITÈRE

L'application permet de rechercher des événements selon trois critères :

- Par titre — recherche partielle insensible à la casse (LIKE + LOWER)
- Par date — recherche par date exacte au format YYYY-MM-DD
- Par type — recherche par catégorie d'événement

FONCTIONNALITÉS PRINCIPALES

```
private void rechercher() {
    String motCle = txtRecherche.getText().trim();
    if (motCle.isEmpty()) {
        alerte(Alert.AlertType.WARNING, "Entrez un mot clé pour rechercher.");
        return;
    }
    try {
```

```
        List<Evenement> resultats;
        switch (comboRecherche.getValue()) {
            case "Date":
                if (!validerDate(motCle)) {
                    alerte(Alert.AlertType.WARNING, "Format de date invalide. Utilisez YYYY-MM-DD.");
                    return;
                }
                resultats = dao.rechercherParDate(motCle);
                break;
            case "Type":
                resultats = dao.rechercherParType(motCle);
                break;
            default:
                resultats = dao.rechercherParTitre(motCle);
        }
        if (resultats.isEmpty()) {
            alerte(Alert.AlertType.INFORMATION, "Aucun événement trouvé.");
        }
        table.setItems(FXCollections.observableArrayList(resultats));
    } catch (SQLException ex) {
        alerte(Alert.AlertType.ERROR, "Erreur: " + ex.getMessage());
    }
```

3 . CHARGERTOUS

```
private void chargerTous() {
    try {
        List<Evenement> list = dao.afficherTous();
        table.setItems(FXCollections.observableArrayList(list));
    } catch (SQLException ex) {
        alerte(Alert.AlertType.ERROR, "Erreur: " + ex.getMessage());
    }
}
```

4 . AFFICHERIMPORTANTS

```
private void afficherImportants() {
    try {
        List<Evenement> list = dao.afficherImportants();
        if (list.isEmpty()) {
            alerte(Alert.AlertType.INFORMATION, "Aucun événement important trouvé.");
        }
        table.setItems(FXCollections.observableArrayList(list));
    } catch (SQLException ex) {
        alerte(Alert.AlertType.ERROR, "Erreur: " + ex.getMessage());
    }
}
```

Cette méthode appelle le DAO pour récupérer les événements importants et les affiche dans le TableView, avec un message si aucun résultat n'est trouvé.

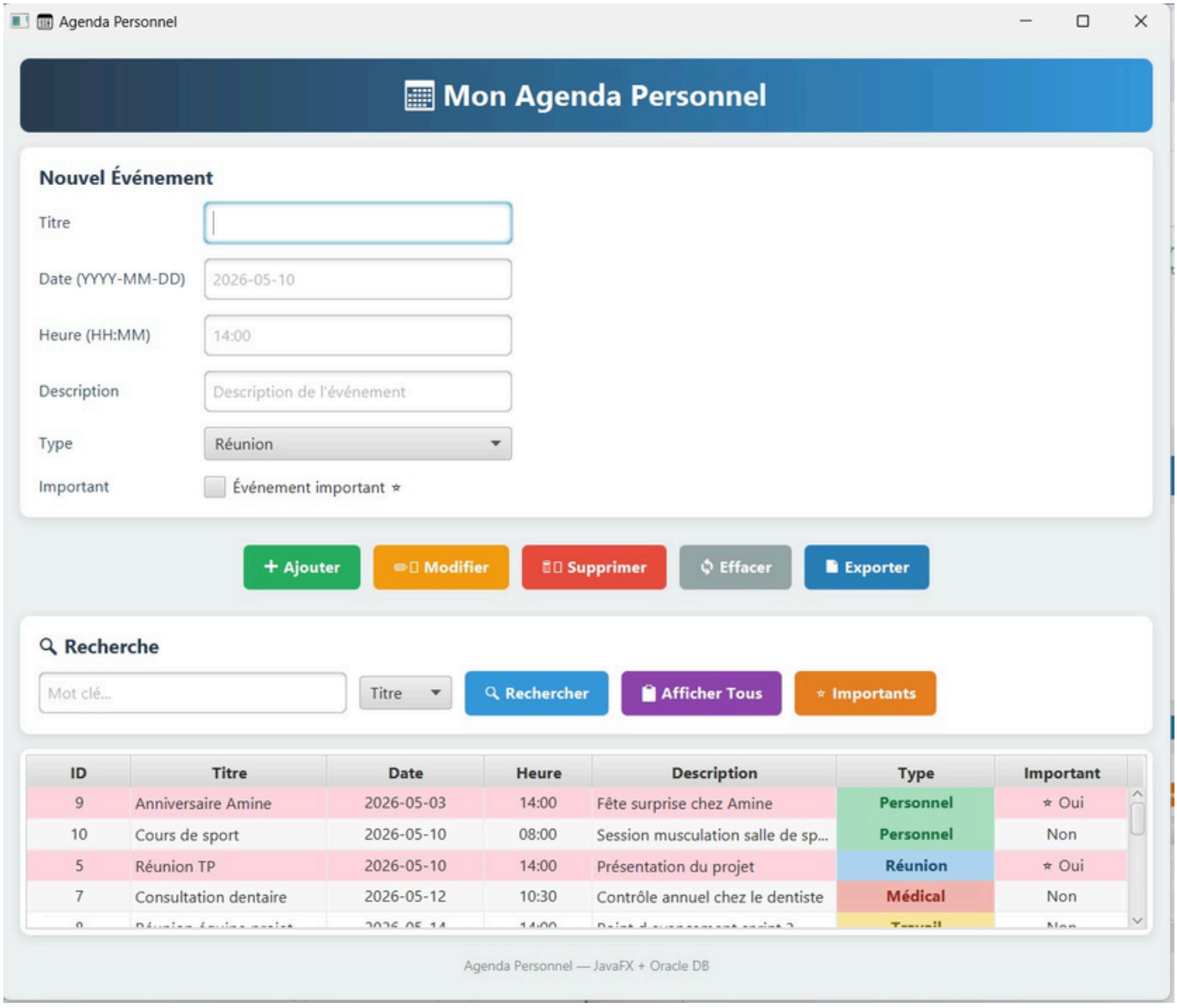
5 . EXPORT EN FICHIER TEXTE

```
private void exporterEvenements() {
    try {
        List<Evenement> liste = dao.afficherTous();
        if (liste.isEmpty()) {
            alerte(Alert.AlertType.WARNING, "Aucun événement à exporter !");
            return;
        }
        String nomFichier = "agenda_export.txt";
        dao.exporterFichier(nomFichier, liste);
        alerte(Alert.AlertType.INFORMATION, "Export réussi !\nFichier : " + nomFichier);
    } catch (Exception ex) {
        alerte(Alert.AlertType.ERROR, "Erreur d'export : " + ex.getMessage());
    }
}
```

INTERFACE GRAPHIQUE MODERNE

L'interface utilise JavaFX avec un design moderne incluant :

- En-tête avec dégradé bleu (linear-gradient)
- Boutons colorés avec effet hover
- Cartes avec ombres portées (dropshadow)
- Tableau avec colonnes centrées et redimensionnables
- Fond gris clair (#ecf0f1) pour un rendu professionnel



MÉTHODES UTILITAIRES (HELPERS UI)

Ces méthodes permettent de créer des composants graphiques stylisés (TextField, Label, Button) et d'afficher des messages d'alerte pour améliorer l'interface utilisateur.

```
// === HELPERS UI ===

private TextField createStyledTextField(String prompt) {
    TextField tf = new TextField();
    tf.setPromptText(prompt);
    tf.setStyle("-fx-font-size: 13px; -fx-pref-width: 250; -fx-background-radius: 5; -fx-border-radius: 5; -fx-border-
    return tf;
}

private Label createLabel(String text) {
    Label lbl = new Label(text);
    lbl.setFont(Font.font("Segoe UI", FontWeight.SEMI_BOLD, 13));
    lbl.setStyle("-fx-text-fill: #34495e;");
    return lbl;
}

private Button createStyledButton(String text, String color) {
    Button btn = new Button(text);
    btn.setStyle("-fx-background-color: " + color + "; -fx-text-fill: white; -fx-font-size: 13px; -fx-font-weight: bol
    btn.setOnMouseEntered(e -> btn.setStyle("-fx-background-color: derive(" + color + ", -15%); -fx-text-fill: white;
    btn.setOnMouseExited(e -> btn.setStyle("-fx-background-color: " + color + "; -fx-text-fill: white; -fx-font-size:
    return btn;
```

MÉTHODE : ALERTE() – AFFICHAGE DES MESSAGES UTILISATEUR

```
private void alerte(Alert.AlertType type, String message) {
    Alert alert = new Alert(type, message);
    alert.showAndWait();
}
```

Conclusion :

Ce travail pratique nous a permis d'appliquer les concepts de la programmation Java à travers une application concrète de gestion d'événements.
L'agenda développé offre une interface simple, une base de données fiable et des contrôles assurant la validité des données.
Les principaux apports sont :

- utilisation de JDBC avec Oracle
- gestion des exceptions
- utilisation des expressions lambda
- export des données via fichiers
- création d'une interface avec JavaFX
- ajout d'une fonctionnalité pour afficher les événements importants

Des améliorations possibles incluent l'ajout de notifications, d'une vue calendrier ou d'une fonction d'import.